

Typed assembly for worst-case execution time analysis

Marcus Janum

May 28, 2026

$r ::=$	$r1 \mid \dots \mid rk$	<i>registers:</i>	$\iota ::=$	$r_d := v$	<i>instructions:</i>
$v ::=$	n	<i>operands:</i>	$r_d := r_s + v$	$\text{if } r \text{ jump } vv$	
	ℓ	<i>integer literal</i>			<i>instruction sequences:</i>
	r	<i>label or pointer</i>	$I ::=$	$\text{jump } v$	
		<i>registers</i>		$\iota; I$	

Figure 1: instructions and operands for the `tal0` language

1 Introduction

2 Typed Assembly Languages

Typed assembly languages are a group of languages (not unlike assembly) or rather just an idea that in varying degrees introduce type safety to programs written or compiled to assembly. The primary motivation for typed assembly languages is type safety and is strongly related to proof-carrying code [4, ch. 5].

In this section we will cover the canonical reason for introducing types as well as exploring levels of typing and implementations. Most of this section is based on Greg Morrisett's work whom has contributed most of the published research in this topic.

2.1 Why type safety in assembly?

Running untrusted code

2.2 TAL-0

As with PCC, it is of great interest to prove correctness of code. In modern programming languages this is done in part with a strongly typed syntax and semantics as “well-typed programs cannot ‘go wrong’”, thus moving some of the burden of correctness to the type system. The goal of TAL-0 (besides being a paedological tool from Morrisett to introduce readers to typed assembly) is to restrict control-flow to ensure control-flow safety of assembly code.

An example of a potential mishap — and how it may look solved — may be as the following risc-v snippet; jumping to the contents of a register, *hopefully* an address.

```
jump :
    jr      a0
```

This short example showcases how naive the machine is, as no checks need be performed, and no assurance is made, that ‘a0’ does indeed contain a valid code address. `tal0` solves this by special syntax and an abstract machine that only allows some evaluation; the above assembly snippet would be disallowed by the evaluation rules.

By figure 1, the previous code snippet would look like

```
jump :
    jump    r1
```

2.2.1 The abstract machine

The abstract machine presented here is in accordance to Morrissets TAL-0. The machine presented here will be extended in future sections.

2.2.2 Typing rules

$ftv(\tau)$ is the set of free variables for τ . Not sure where it looks to determine free-ness? a variable x is bound in τ if $\forall x.\tau$, but i think this may be a writing mistake? Surely it should be like $ftv(\forall x.y) = \{y\}$ while $ftv(\forall x.x) = \{\emptyset\}$

2.2.3 Soundness

The proof relies on the abstract machine not getting stuck.

3 worst-case execution time analysis

Worst-case execution time analysis is an important step in the process of developing real-time systems [5] like `ssd_os`. In general, and is easily exemplified, not all programs are subject to execution time analysis, nor do all programs exit. The former is a consequence of the halting problem [7, ch. 13] [5].

3.1 Observed observation time

end-to-end measurements

3.2 static analysis

4 `ssd_os` (The system)

We are not really considering a heap at all (even though obviously program text exists, and `.words` can exist there, too).

4.1 Model

This project specifically targets the cacheless v80 platform [1]. It consists of four clusters with four AMD MicroBlaze V processors, each having its own scratchpad memory, and all clusters connected to a shared scratchpad, referred to as shared moving forwards. The key take-away is that shared memory is slower than scratch memory.

4.2 Allocating

```

struct memory_region
{
    uint8_t *start;
    size_t   size;
};

struct arena_allocator
{
    struct memory_region region;
    size_t               allocated;
};

```

Figure 2: Structure of an allocator

```

uint8_t *arena_alloc(struct arena_allocator *alloc, size_t size, size_t
    alignment)
{
    uint8_t      *free_region_start = alloc->region.start + alloc->allocated;
    const size_t alignment_padding =
        (alignment - (((size_t) free_region_start) % alignment)) %
        alignment;
    const size_t size_to_alloc = size + alignment_padding;

    const size_t remaining =
        (alloc->region.start + alloc->region.size) - free_region_start;

    if (remaining < size_to_alloc)
        return NULL;

    uint8_t *allocated = free_region_start + alignment_padding;
    alloc->allocated += size_to_alloc;
    return allocated;
}

```

Figure 3: Example of an allocator function

5 Possibilities

5.1 Control flow

One of the motivations behind typed assembly languages as presented earlier (TODO: REF SECTION) is control flow. TAL-0 introduces types to code to restrict where jumps are allowed to target. Control-flow in regards to execution time analysis is not so interested in the validity of jumps but on the amount of jumps, i.e. loops.

Control-flow graph

By using DTAL and considering a program an oriented graph where blocks are nodes and branches are edges to create something like a control-flow graph.

5.2 Typing memory

The chosen platform is explicit in its memory hierarchy, which enables it to be explicit in code as well. This can be done through a type system, so any array or stack type is wrapped inside a memory region context.

5.3

6 Getting to a language

This section describes the process, the steps, and the considerations for designing the risc-v styled typed assembly language suited for static worst-case execution time analysis. Only a subset of the idea of the language will be fully fleshed out, and an even smaller part implemented. The “full” language requires much too complex typing that involves constraint satisfaction of predicate logic, far out of the scope of this project, unfortunately. “The language” refers to the implementation TAL designed for WCET. The language syntax is pictured in figure 4. All registers map to τ , except when otherwise noted. The register names follow the risc-v ABI names [3]. If a register is not explicitly typed, it will default to top . The jump instruction j is kept only to link blocks together, mimicking assembly fall-through.

6.1 Strays from risc-v

As with the TALs introduced above, there is little reason to distinguish between immediate instructions and register instructions, meaning that subi and sub can be treated the same. This is due to the value type that they all share in some form, defined in some way as

$$\text{value } v ::= c \mid r$$

For code generation, these will have to be reintroduced, but this is trivial, since the value field will either be an immediate value or a register.

Since all types are word sized, we replace the load and store instructions with $\text{load } r_d, \text{imm}(r_s)$ and $\text{store } r_s, \text{imm}(r_d)$

Two very specific instructions are also added: mvscr and mvshr . They serve a very specific purpose in moving the start address of either the scratch or shared memory regions into register r_d . Initially I did not want to stray from risc-v, so using a combination of lui and addi , however that would create an incomplete type between the two instructions. For a while, the second operand of mv could be a value type v , which would then be translated to lui and addi

type variables	α	
State type	σ	$::= \lambda \Delta. R$
Register files	R	$::= [\mathbf{zero} : \mathit{int}, \mathbf{ra} : \tau_1, \mathbf{sp} : S, \dots]$
region memory types variables	μ	
region memory types	M	$::= \mathit{scratch} \mid \mathit{shared} \mid \mu$
stack types variables	ρ	
stack types	S	$::= [] \mid \rho \mid \tau :: S$
Collection types	C	$::= M \tau \mathit{array}(i) \mid M S$
types	τ	$::= \alpha \mid \mathbf{top} \mid \mathit{int} \mid \mathit{int}(i) \mid C$
type variable contexts	Δ	$::= \cdot_{\mathbf{tv}} \mid \Delta, \mu : M \mid \Delta, \rho : S \mid \Delta, \alpha : \tau$
integer literals	i	
constants	c	$::= i \mid l$
values	v	$::= c \mid r \mid C$
labels	l	
instructions	ins	$::= aop\ r_d,\ r_s,\ v$ $\mid \mathbf{incr}\ r_d,\ r_s,\ v$ $\mid \mathbf{decr}\ r_d,\ r_s,\ v$ $\mid \mathbf{mv}\ r_d,\ r_s$ $\mid \mathbf{j}\ v$ $\mid \mathbf{halt}$ $\mid \mathbf{mvscr}\ r_d$ $\mid \mathbf{mvshr}\ r_d$
arithmetic operations	aop	$::= \mathbf{add} \mid \mathbf{sub} \mid \mathbf{mul} \mid \mathbf{div}$
instruction sequences	I	$::= \mathbf{j}\ v \mid \mathbf{halt} \mid ins;\ I$
Blocks	B	$::= \lambda \Delta. (R, I)$
Label mappings	Λ	$::= \{l_1 : \sigma_1, \dots, l_n : \sigma_n\}$
Programs	P	$::= l_1 : B_1; \dots; l_n : B_n$

Figure 4: Syntax for a typed assembly language.

```

unsigned int *simple_malloc(struct arena_allocator *alloc, size_t size)
{
    unsigned int* start = alloc->region.start + alloc->allocated;
    alloc->allocated += size;
    return start;
}

```

Figure 5: A simple allocation function

if the type of v called for it. But still, magic values for the memory region pointers were needed. This is the problem `mvscr` and `mvshr` attempt to solve, however ugly it may be.

6.2 Types

While the syntax of the language is inspired by DTAL, it is much simplified (or mangled, even). This is also the case of the τ types, where only a , top , int , and $\text{int}(i)$ remain. The array and stack types are still present but wrapped, but collections will be explained later. Universally quantified types, i.e. type variables are still present, although they do not serve a concrete function. Stack type variables are still present. They are needed to append to a stack, which is needed for `incr`. Specific for the language, region memory type variables have been introduced. They allow labels to be polymorphic over regions. This is an ergonomic must, as otherwise the programmer (or a compiler) would have to write labels for both `scratch` and `shared`.

For the register file, this also means that any type that is not explicitly typed defaults to `top`. This means that for later blocks, `top` can freely be coerced into a new type.

The dependent type $\text{int}(x)$ where x is an index expression has been reintroduced as $\text{int}(i)$ to allow static evaluation of stack types in `incr` and `decr`. The instructions need to add the right amount of types to the head of the stack. It follows that they should only allow immediate values. This is reflected in their evaluation rules and typing rules in figure 8 and figure 9, respectively.

The type variable context strays from the inspiration (DTAL) by essentially a set of bindings. The inspiration comes from the hindley-milner algorithm [7]. This choice also has a big implication on the type equality and coercion rules, as these now produce a side-effect, i.e. they change or add mappings to the context Δ . Another implication of this is that it forces the entrypoint to have no type variables. This makes sense, as there would be no way to deduce the type information. If for some reason one wanted to explicitly state “anything” one would have to use `top`. If $\alpha \in \text{dom}(\Delta)$ then $\Delta\alpha = \tau$, and similarly for μ and ρ .

6.3 Collections

This section describes how the array and stack types work with each other, and how they are to be thought of conceptually in the language moving forwards. Some TALs introduce `malloc` or similar pseudo-instructions to introduce a notion of types to arrays. Since assembly does not in of itself contain arrays, but only pointers, some level of abstraction needs to be introduced for a type system to be able to type arrays and differentiate them from garbage and labels. In C, heap arrays are pointers, and pointer arithmetic is allowed. Labels can only be used in `goto` statements, and not even arithmetic is permitted on them. Still, it is not trivial whether `float* n` points to a scalar, or is an array.

A malloc instruction is not suited in this case, since the target system contains multiple allocators (as discussed in section TODO: REF HERE, THE SYSTEM) working over memory regions. The problem then evolves into the question of how to design a type system that makes an allocator implicitly emit an array type, or a type that can be coerced into arrays. A simple allocator is shown in figure 5. It has no safety checks, and it does not align allocated memory. The TAL type systems often only accept word sized types, and for simplicity, so will I. To answer this, we first look into stacks, arrays, and a type system. Another issue is the fact that the stack pointer grows downwards, while the arena stacks grow upwards. This will also be solved.

6.3.1 Stacks

Stack types can be defined recursively as

$$\begin{array}{ll} \text{stack types variables} & \rho \\ \text{stack types} & S ::= [] \mid \rho \mid \tau :: S \end{array}$$

[6]

In DTAL, the stack type is a part of the type state Σ , and is modified through pushd and pop operations, a very conventional way to use the stack. Here, stacks are used not only for the call-stack, but general allocations, and to be coerced into array types. To communicate to the type system that we intend to modify a stack, we introduce two instructions incr and decr. They may be described through the code generation in 14. incr decreases the stack pointer in r_s by v words (removing the need for runtime alignment) and places the value in r_d whose types will be that of r_s with v top values appended to it, e.g.,

```
main:    ('r)
         [sp: 'r]
         incr    a0, sp, 4    ; a0 : top :: top :: top :: top :: 'r
```

where 'r is a stack type variable. decr functions as expected, popping v words off the stack.

A programmer may not be interested in passing on *the whole* stack after increasing the word count, such as stack arrays in C. If the type of a stack allocated array was the whole call-stack, not only would it obfuscate the intend, it would prevent memory safety (if the type system enforced such things). It is therefore desirable to “cut down” or slice a stack. This can be done by considering a stack S a subtype of a stack S' if S is contained within S' with a possibly empty tail. Or formally

$$\tau_0 :: \dots :: \tau_n \leq \tau_0 :: \dots :: \tau_m \mid n \leq m$$

To illustrate the usefulness, we can expand on the previous code snippet:

```
main:    ('r)
         [sp: 'r]
         incr    a0, sp, 4    ; a0 : top :: top :: top :: top :: 'r
subtype: [a0: top :: top :: top :: top]
         ..
```

While top is not so useful in of itself since it practically refers to garbage values, we will later introduce a subtyping rule to top, specifically such that any type can be coerced from top. This makes it possible to instantiate a stack of any type (although the language may not support so many types) without introducing special “typed” syntax to incr and decr, as DTALs `newarray[ $\tau$ ]`. So really we could write:


```

start: [a0: int, a1: int, a2: int, a3: int]
alloc: [a0: int, a1: int, a2: int, a3: int]
      incr    a4, sp, 4      ; a4: top :: top :: top :: top :: ..
store: ('m)
      [a0: int, a1: int, a2: int, a3: int, a4: 'm int array(4)]
      store   a0, 0(a4)
      store   a1, 1(a4)
      store   a2, 2(a4)
      store   a3, 3(a4)

```

Figure 6: Example code for allocating a stack array and writing to it

```

main:   ('r)
        [sp: 'r]
        incr    a0, sp, 4    ; a0 : top :: top :: top :: top :: 'r
subtype:('a)
        [a0: 'a :: 'a :: 'a :: 'a]
        ..

```

for some type variable 'a.

6.3.2 Arrays

Since we have foregone array initialization instructions, we add the choice to coerce a stack containing only one type to an array of that type. This models C stack arrays quite well.

6.3.3 Array allocation using stacks

At this point we can subtype stacks to create smaller stacks, and we can instantiate array types by coercing a stack.

While the call-stack grows downwards and `incr` and `decr` can be used as expected, the memory region allocations grows upwards. The solution lies in the conceptualization.

While `incr` and `decr` only work on immediate values here, this is only for simplification, and since the language contains no looping constructs, there is no value in having dependent sizes.

An example program that allocates a stack array and stores four integers could look like the program in figure 6.

Another use is working with the arena allocators. They are simple, and also work like a stack, but these grow upwards, which is opposite the call stack and therefore the typing rules would not work.

```

struct memory_region
{
    uint8_t *start;
    size_t   size;
};

struct arena_allocator
{
    struct memory_region region;
    size_t               allocated;
}

```

```

 $\Delta_0$  =  $\cdot_{\text{tv}}$ 
 $\Delta_1$  =  $\text{'sp}$ 
 $\Delta_2$  =  $\text{'m}$ 
 $R_0$  =  $\text{a0 : int, a1 : int, a2 : int, a3 : int,}$ 
 $R_1$  =  $\text{a0 : int, a1 : int, a2 : int, a3 : int, sp : 'sp}$ 
 $R_2$  =  $\text{a0 : int, a1 : int, a2 : int, a3 : int, a4 : 'm int array(4)}$ 
 $I_0$  =  $\text{j alloc}$ 
 $I_1$  =  $\text{incr a4, sp, 4}$ 
        $\text{j store}$ 
 $I_2$  =  $\text{store a0, 0(a4)}$ 
        $\text{store a1, 1(a4)}$ 
        $\text{store a2, 2(a4)}$ 
        $\text{store a3, 3(a4)}$ 
        $\text{halt}$ 
 $B_0$  =  $\lambda\Delta_0.(R_0, I_0)$ 
 $B_1$  =  $\lambda\Delta_1.(R_1, I_1)$ 
 $B_2$  =  $\lambda\Delta_2.(R_2, I_2)$ 
 $P$  =  $\text{start : } B_0; \text{ alloc : } B_1; \text{ store : } B_2$ 

```

Figure 7: Explanation of code in figure 6

```
};
```

While the purpose of the language is only typed memory regions,

6.4 Abstract machine

Unlike other TAL abstract machines, I am omitting the stack, since it will be used as a normal register (sp). The DTAL abstract machine has a machine state $\mathcal{M}(\mathcal{H}, \mathcal{R}, \mathcal{S})$ being a triple over a heap, register file, and stack. Since the system the language is targeted at is stack based, no heap is needed (except for program text, but this is omitted partially due to program scope) and no explicit stack is needed, since the register sp will be used. As such, the abstract machine contains only a finite mapping between registers and values.

This is also the motivation behind removing the DTAL type state Σ and state type σ .

6.5 Dynamic Semantics

Dynamic semantics is needed to evaluate the types throughout block execution for reasons discussed in section (TODO: REF IDEAS – WHY NOT TRANSITIONS).

Since we are explicit in the use of sp the abstract machine does not need to keep track of this. The heap is also omitted. We instead refer to hardcoded values: the start of each memory region. This means that

```

start:
    mv    a0, SCRATCH_START

```

will set the type of a0 in the register file to `scratchS`, where `SCRATCH_START` refers to an immediate value that points to the start of the scratch region. An abstract machine state $\mathcal{M}$ naturally reduces to just the register file. We define $\mathcal{M} = \mathcal{R}$ where $\mathcal{R}(r)$ is a constant.

$$\begin{array}{c}
\frac{J(ic) = \text{add } r_d, r_s, v \quad \mathcal{M}(r_s) = i \quad \mathcal{M}(v) = j}{(ic, \mathcal{R}) \rightarrow_P (ic + 1, \mathcal{R}[r_d = i + j])} \text{ (eval-add)} \\
\frac{J(ic) = \text{incr } r_d, r_s, v \quad \mathcal{M}(r_s) = \mu S \quad \mathcal{M}(v) = i \quad i > 0}{(ic, \mathcal{R}) \rightarrow_P (ic + 1, \mathcal{R}[r_d = \mu \text{top}_0 :: \dots :: \text{top}_i :: S])} \text{ (eval-incr)} \\
\frac{J(ic) = \text{incr } r_d, r_s, v \quad \mathcal{M}(r_s) = \mu \text{top}_0 :: \dots :: \text{top}_i :: S \quad \mathcal{M}(v) = i \quad i > 0}{(ic, \mathcal{R}) \rightarrow_P (ic + 1, \mathcal{R}[r_d = \mu S])} \text{ (eval-decr)}
\end{array}$$

Figure 8: Evaluation rules

Similar to the treatment in DTAL, we introduce an instruction sequence J_P of a program P , and I_B for the instruction sequence in B as

$$J_P = l_1; I_{B_1}; \dots; l_n; I_{B_n}$$

We use $J_P(i)$ to refer to the i th instruction or label in J_P . If it is clear what P is being referred to, we may omit the subscript. We also use $J_P^{-1}(l) = i \iff J_P(i) = l$ from DTAL.

A P -snapshot [6] is either `halt` or a tuple $(ic, \mathcal{M})$ such that $0 \leq ic < \text{length}(J_P)$, where ic is the instruction counter. $(ic, \mathcal{M}) \rightarrow_P (ic', \mathcal{M}')$ means that the machine state $\mathcal{M}$ transforms into $\mathcal{M}'$ by executing the instruction at $J_P(ic)$. Similarly ic' is incremented or replaced in the case of `j`.

$$\mathcal{M}(v) = \begin{cases} i & \text{if } v \text{ is integer } i \\ l & \text{if } v \text{ is label } l \\ C & \text{if } v \text{ is collection } C \\ \mathcal{R}(r) & \text{if } v \text{ is register } r \end{cases}$$

Similar to DTAL.

6.6 Static Semantics

The typing rules for the language is the most important step in the solution of calculating worst-case execution time with the method discussed.

One difference between other TALs and especially how values v in DTAL and how they behave here, is that we need a way to instantiate the region memory types. Earlier instantiating was presented as

In figure 10 presenting the typing rules, ρ_{scratch} and ρ_{shared} represent two stack types. Each region should be defaulted to `top`, and the amount depends on the physical size. Since the scratchpad is 128KB [1], ρ_{scratch} may be defined as $\text{top}_1 :: \dots :: \text{top}_{128/w \cdot 1000} :: []$ where w is the machines wordsize.

The type rules for values in figure 9 depend on the label mapping Λ for typing labels. This is the meaning of the notation $\vdash_\Lambda$, where Λ is a label mapping function. In practice, only one label mapping function will be used, so for brevity $\vdash$ may be used — like in DTAL.

The typing rules for arithmetic instructions follow from `add` in figure 10.

The typing for jumps `j` do not continue checking the instructions of the target block. It checks if the current state aligns with the state type σ in the target block. This means comparing each register type. This is explained in section 6.7. Since the handling of stacks and arrays rely heavily on subtyping, we use the coercion rules for comparison or acceptance of a new state type.

The type rules differ from DTAL in that they use and (potentially) *update* the type variable context Δ . This is specifically for variable typing rules. The motivation for this choice is jumps,

$$\begin{array}{c}
\frac{}{\Delta; R \vdash_{\Lambda} i : \text{int}(i)} \text{ (type-immediate-int)} \\
\frac{\Delta(l) = \sigma}{\Delta; R \vdash_{\Lambda} l : \sigma} \text{ (type-label)} \\
\frac{}{\Delta; R \vdash_{\Lambda} r : R(r)} \text{ (type-reg)} \\
\frac{\Delta(\alpha) = \tau}{\Delta; R \vdash_{\Lambda} \alpha : \tau} \text{ (type-var)} \\
\frac{\Delta(\rho) = S}{\Delta; R \vdash_{\Lambda} \rho : S} \text{ (type-stack-var)} \\
\frac{\Delta(\mu) = M}{\Delta; R \vdash_{\Lambda} \mu : M} \text{ (type-memory-var)}
\end{array}$$

Figure 9: Typing rules for values

going from one machine state to another. To make use of the ergonomics discussed earlier in regards to handling stacks and arrays, a machine state M consisting of a register file and variable mapping can jump to M' iff M can be coerced into M' . Or in other words

$$\Delta, R \models_c R' \rightsquigarrow \Delta'$$

When adding a binding to a type variable context, we do not want to accidentally define circular types such as $\alpha : \beta, \beta : \alpha$. This is why the variable type equality and coercion do not have rules for other variables, and as a consequence, type variable contexts can only have a depth of one, if one were to imagine Δ as a graph. This is not dissimilar in the treatment of existential qualifiers [2], in the sense of a graph.

Because of this constraint type variables cannot refer to other type variables, when we coerce register files we can assert that at type-coerce-regfile, type-reg will never evaluate to a type variable. This means that there is no need to check if variables in the two register files are unique, or in other words, we do not have to create fresh variables, something that is required for hindley-milner type inference [7].

6.7 Type Equality and Coercion

We start by introducing the type equality and coercion rules.

The typing rules for collection types contain a lowercase c , which is taken to mean the pattern matched type of either $\tau \text{ array}(i)$ or S .

The coercion rules allow the type variable context to be modified, by coercing type variables into types. The use case is limited to state transitions, i.e. jumps. This is the part that binds type variables to types, specifically

$$\frac{\Delta; R \vdash r_n : \tau_n \quad \Delta_{n-1}; R' \vdash r_n : \tau'_n \quad \cdot_{\text{tv}} \models \tau'_n \leq \tau_n \rightsquigarrow \Delta_n}{\Delta; R \models_c R' \rightsquigarrow \Delta_n}$$

where Δ_n is created through accumulating type captures starting at $\Delta_1 = \cdot_{\text{tv}}$ for all n registers.

Since I decided to reflect the risc-v register ABI names in the syntax of the language, it is not actually possible to iterate over all registers. Not for any semantic reasons. Therefore I informally introduce r_n to be a register, and the above meaning is that it iterates over all (n) registers.

$$\begin{array}{c}
\frac{\Delta; R \vdash r_s : \text{int} \quad \Delta; R \vdash v : \text{int} \quad \Delta; R \vdash I}{\Delta; R \vdash \text{add } r_d, r_s, v; I} \text{ (type-add)} \\
\frac{\Delta; R \vdash r_s : \mu S \quad \Delta; R \vdash v : \text{int}(i) \quad i > 0 \quad \Delta; R[r_d = \mu \text{top}_1 :: \dots :: \text{top}_i :: S] \vdash I}{\Delta; R \vdash \text{incr } r_d, r_s, v; I} \text{ (type-incr)} \\
\frac{\Delta; R \vdash \text{decr } r_d, r_s, v; I \quad \Delta; R \vdash v : \text{int}(i) \quad i > 0 \quad \Delta; R[r_d = \mu S] \vdash I}{\Delta; R \vdash r_s : \mu \tau_1 :: \dots :: \tau_i :: S} \text{ (type-decr)} \\
\frac{\Delta; R \vdash \text{mvshr } r_d; I \quad \Delta; R[r_d = \tau] \vdash I}{\Delta; R \vdash r_s : \tau} \text{ (type-mv)} \\
\frac{\Delta; R \vdash \text{mvshr } r_d; I}{\Delta; R[r_d = \text{scratch } \rho_{\text{scratch}}] \vdash I} \text{ (type-mvscratch)} \\
\frac{\Delta; R \vdash \text{mvshr } r_d; I}{\Delta; R[r_d = \text{shared } \rho_{\text{shared}}] \vdash I} \text{ (type-mvshared)} \\
\frac{\Delta; R \vdash v : \lambda_{\text{tv}} . R' \quad \Delta; R \models_c R' \rightsquigarrow \Delta'}{\Delta; R \vdash \text{j } v} \text{ (type-jump)}
\end{array}$$

Figure 10: Typing rules

$$\begin{array}{c}
\frac{\alpha \in \Delta}{\Delta \models \alpha \equiv \alpha} \text{ (type-eq-var)} \\
\frac{}{\Delta \models \text{top} \equiv \text{top}} \text{ (type-eq-top)} \\
\frac{}{\Delta \models \text{int} \equiv \text{int}} \text{ (type-eq-int)} \\
\frac{\Delta \models \tau \equiv \tau' \quad \Delta \models S \equiv S'}{\Delta \models \tau :: S \equiv \tau' :: S'} \text{ (type-eq-stack)} \\
\frac{}{\Delta \models [] \equiv []} \text{ (type-eq-emptystack)} \\
\frac{\Delta \models \tau \equiv \tau' \quad i = j}{\Delta \models \tau \text{array}(i) \equiv \tau' \text{array}(j)} \text{ (type-eq-array)} \\
\frac{}{\Delta \models \text{scratch} \equiv \text{scratch}} \text{ (region-eq-scratch)} \\
\frac{}{\Delta \models \text{shared} \equiv \text{shared}} \text{ (region-eq-shared)} \\
\frac{\Delta \models \mu \equiv \mu' \quad \Delta \models c \equiv c'}{\Delta \models \mu c \equiv \mu' c'} \text{ (type-eq-collection)}
\end{array}$$

Figure 11: Type equality rules

$$\begin{array}{c}
\frac{\alpha \in \mathbf{dom}(\Delta)}{\Delta \models \alpha \leq \alpha \rightsquigarrow \Delta} \text{ (type-coerce-var)} \\
\frac{\alpha \notin \mathbf{dom}(\Delta)}{\Delta \models \alpha \leq \text{int} \rightsquigarrow \Delta[\alpha: \text{int}]} \text{ (type-coerce-var-int)} \\
\frac{\alpha \notin \mathbf{dom}(\Delta)}{\Delta \models \alpha \leq \text{int}(i) \rightsquigarrow \Delta[\alpha: \text{int}(i)]} \text{ (type-coerce-var-imm)} \\
\frac{\alpha \notin \mathbf{dom}(\Delta)}{\Delta \models \alpha \leq \text{int} \rightsquigarrow \Delta[\alpha: \text{int}]} \text{ (type-coerce-var-collection)} \\
\frac{\Delta(\alpha) = \tau' \quad \Delta \models \tau \equiv \tau'}{\Delta \models \alpha \leq \tau \rightsquigarrow \Delta} \text{ (type-coerce-var-t)} \\
\frac{\rho \notin \mathbf{dom}(\Delta)}{\Delta \models \rho \leq \rho \rightsquigarrow \Delta} \text{ (type-coerce-stack-var)} \\
\frac{\rho \notin \mathbf{dom}(\Delta)}{\Delta \models \rho \leq [] \rightsquigarrow \Delta[\rho: []]} \text{ (type-coerce-stack-var-empty)} \\
\frac{\rho \notin \mathbf{dom}(\Delta) \quad \Delta \models S \rightsquigarrow \Delta'}{\Delta \models \rho \leq \tau::S \rightsquigarrow \Delta[\rho: \tau::\Delta'(\rho)]} \text{ (type-coerce-stack-var-cons)} \\
\frac{\mu \in \mathbf{dom}(\Delta)}{\Delta \models \mu \leq \mu \rightsquigarrow \Delta} \text{ (type-coerce-memory-var)} \\
\frac{\mu \notin \mathbf{dom}(\Delta)}{\Delta \models \mu \leq \text{shared} \rightsquigarrow \Delta[\mu: \text{shared}]} \text{ (type-coerce-memory-var-shared)} \\
\frac{\mu \notin \mathbf{dom}(\Delta)}{\Delta \models \mu \leq \text{scratch} \rightsquigarrow \Delta[\mu: \text{scratch}]} \text{ type-coerce-memory-var-scratch} \\
\frac{}{\Delta \models \tau \leq \text{top} \rightsquigarrow \Delta} \text{ (type-coerce-top)} \\
\frac{}{\Delta \models \text{int} \leq \text{int} \rightsquigarrow \Delta} \text{ (type-coerce-int)} \\
\frac{\Delta \models \tau_1^l \equiv \tau_1^r \quad \dots \quad \Delta \models \tau_n^l \equiv \tau_n^r}{\Delta \models \tau_1^l::\dots::\tau_n^l::S \equiv \tau_1^r::\dots::\tau_n^r::S' \rightsquigarrow \Delta} \text{ (type-coerce-stack)} \\
\frac{\Delta \models \tau \text{ array}(i) \equiv \tau' \text{ array}(j)}{\Delta \models \tau \text{ array}(i) \leq \tau' \text{ array}(j) \rightsquigarrow \Delta} \text{ (type-coerce-array)} \\
\frac{\Delta \models \tau \equiv \tau_0 \quad \dots \quad \Delta \models \tau \equiv \tau_i}{\Delta \models \tau \text{ array}(i) \leq \tau_0::\dots::\tau_i::S} \text{ (type-coerce-array-stack)} \\
\frac{\Delta \models \mu \equiv \mu' \quad \Delta \models c \leq c'}{\Delta \models \mu c \leq \mu' c'} \text{ (type-coerce-collection)} \\
\frac{\Delta; R \vdash r_n : \tau_n \quad \Delta_{n-1}; R' \vdash r_n : \tau'_n \quad \Delta_{n-1} \models \tau'_n \leq \tau_n \rightsquigarrow \Delta_n}{\Delta; R \models_c R' \rightsquigarrow \Delta_n} \text{ type-coerce-regfile}
\end{array}$$

Figure 12: Type coercion rules

```

start: [a0: int, a1: int, a2: int, a3: int]
      subi    sp, sp, 16      ; sp: scratch top :: top :: top :: top :: 'r
      mv      a4, sp
coerce: ('m)
      [a0: int, a1: int, a2: int, a3: int, a4: 'm int array(4)]
      lw      a0, 0(a4)      ; a4: scratch int :: ..
      lw      a1, 4(a4)      ; a4: scratch int :: int :: ..
      lw      a2, 8(a4)      ; a4: scratch int :: int :: int ..
      lw      a3, 12(a4)     ; a4: scratch int :: int :: int :: int :: 'r

```

Figure 13: Example program

$$\begin{aligned}
\text{incr } r_d, r_s, v &\mapsto \begin{cases} \text{mul}, & r_d, v, \text{wordsize} & \text{iff } \Phi; \Delta; R \vdash v : \text{is register} \\ \text{sub}, & r_d, r_s, rd & \\ \text{subi}, & r_d, r_s, v \cdot \text{wordsize} & \text{iff } \Phi; \Delta; R \vdash v : \text{is immediate value} \end{cases} \\
\text{decr } r_d, r_s, v &\mapsto \begin{cases} \text{mul}, & r_d, v, \text{wordsize} & \text{iff } \Phi; \Delta; R \vdash v : \text{is register} \\ \text{add}, & r_d, r_s, rd & \\ \text{addi}, & r_d, r_s, v \cdot \text{wordsize} & \text{iff } \Phi; \Delta; R \vdash v : \text{is immediate value} \end{cases}
\end{aligned}$$

Figure 14: Code generation for incr and decr

The coercion rules make it possible to capture type variables and assign them a type. The “instantiation” rules refer to those where the type variable is not already in the variable context; the binding is inserted into the context. It is quite conceivable that the same type variable be used multiple times in the same register file, in this case the coercion type needs to be equal to the type already bound to the type variable in the type variable context.

6.8 Syntax

The syntax for the language is described in figure 4, which takes inspiration from DTAL [6]. As have been discussed, in a complete language with control-flow, type variables should be reintroduced into index expressions.

An example program could look like figure 13

6.9 Code Generation

7 Implementation (wrong title)

7.1 Domain

Only a small subset of what has been discussed will be implemented, namely typing memory region types for the scratchpad system model. The typing rules will ensure that it is known before runtime of which memory region each load/store operation writes or reads from. Since the system works without cache (scratch region effectively taking its place) all IO has a set cost,

and as do the arithmetic instructions. The problem then boils down to essentially summing up the cost of each instruction.

The solution is somewhat novel, since there is a painful lack of control-flow. In the theory description some outline of how this may be thought of and implemented is discussed. The biggest constraint and factor of deciding that control-flow is out of scope is due to the solving process (predicate logic SAT solving).

7.2 The tool

The tool shall take a program in the language specified in section 6, and produce an estimate of its runtime.

From the code it produces a list of instruction and state type tuples, wherein the sum of each instruction time is calculated. For arithmetic instructions this will be constant, and for memory operations it will depend on the memory type. For the pseudo instructions, they will of course take what is equivalent to their code generation.

References

- [1] Philippe Bonnet. σ : An operating system for solid-state drives.
- [2] Michael Huth and Mark Ryan. *Logic in Computer Science: Modelling and Reasoning About Systems*. Cambridge University Press, 2004.
- [3] David A. Patterson and John L. Hennessy. Computer organization and design risc-v edition.
- [4] Benjamin C. Price. Advanced topics in types and programming languages.
- [5] Reinhard Wilhelm, Jakob Engblom, Andreas Ermedahl, Niklas Holsti, Stephan Thesing, David Whalley, Guillem Bernat, Christian Ferdinand, Reinhold Heckmann, Tulika Mitra, Frank Mueller, Isabelle Puaut, Peter Puschner, Jan Staschulat, and Per Stenström. The worst-case execution-time problem—overview of methods and survey of tools. *ACM Trans. Embed. Comput. Syst.*, 7(3), May 2008.
- [6] Hongwei Xi and Robert Harper. A dependently typed assembly language. *SIGPLAN Not.*, 36(10):169–180, October 2001.
- [7] Torben Ægidius Mogensen. Programming language design and implementation.